

Module 7 — Context Engineering

What to write, what to select, what to compress

The model is stateless. Memory is something you build.

Every call sends the whole prompt again — the model remembers nothing on its own.

Memory is two moves: **write** facts to a store, **select** them back into the prompt.

Compaction is the third move, for when history still will not fit.

All three are moves against one budget: the context window.

🎯 Learning goals

- Account for the context window as a **budget** and see where tokens go.
- **Write** memories worth keeping, and **supersede** rather than stack contradictions.
- **Select** by semantic recall, scored on relevance, recency, and importance.
- **Compact** long conversations without losing decisions and open tasks.
- Expose memory and compaction as **MCP tools**.



The context budget

Keeping it under control

Where Context actually goes

System + tools

Persona and policy — fixed cost

Tool schemas grow with every tool you register

Twenty tools can outweigh the conversation

Retrieved docs

The biggest consumer in a RAG agent

Chunk count times chunk size

Reranking here buys budget everywhere else

History

Grows every single turn, forever

Turn 50 resends turns 1-49

The only component with no natural ceiling

Memories

Small — if you select well

Five facts, not fifty

Cheapest component, highest leverage

⚠ Context rot

- A bigger window is not a licence to fill it.
- Accuracy **degrades as the window fills** — distractors crowd out signal.
- Material in the **middle** of a long prompt gets the least attention.
- A tight 2k prompt beats a sprawling 100k one — and costs 2% as much.

Count, do not guess

```
import tiktoken

enc = tiktoken.get_encoding("o200k_base")


def count_tokens(messages: list[dict]) -> int:
    """Approximate the prompt cost of a message list."""
    return sum(
        len(enc.encode(str(m.get("content", "")))) + 4
        for m in messages
    )
```

- → `o200k_base` is the encoding for current OpenAI models (Module 4)
- → Measure each component separately — you cannot budget what you cannot see



Demo — The context budget

Switch to terminal: python demo/demo.py — Part 1



Write

What is worth keeping, and what to do when facts collide

Worth writing: will it matter next week?

Preferences

"I always take the Comet Tail with extra fizz" — shapes every future turn.

Constraints

"I am allergic to synthcitrus" — never stops being true.

Identity

"I am the quartermaster on the Pathfinder" — who you are talking to.

Commitments

"Marek owes 12 credits" — an open loop someone has to close.

Not worth writing

- One-off questions — *"what is on tap tonight?"*
- Transient state — *"I am waiting on my crewmate"*
- Chitchat and pleasantries.
- Anything already in your database — **look it up, do not remember it**.



Select

Recall is a retrieval problem

☹ Why keyed lookup fails

- Early memory systems file by topic and look up by prefix: `recall("drink_")`.
- The patron asks *"what do I usually drink?"*
- The memory is filed under `favourite_beverage`. Nothing matches.
- Embed it and retrieve.

Memories in a vector collection

```
collection = chroma.get_or_create_collection(
    "cantina_memories",
    embedding_function=None,                    # we embed ourselves
    configuration={"hnsw": {"space": "cosine"}},
)

results = collection.query(
    query_embeddings=[embed(query, client)],
    n_results=k * 4,                            # over-fetch, then rescore
    where={"$and": [
        {"patron": {"$eq": patron}},
        {"active": {"$eq": True}},
    ]},
)
```

- The current turn is the query — no keys, no prefixes
- `patron` is the **namespace**: the `where` filter is what stops memory leaking between users
- Over-fetch from the index so there is something to rescore

Score: relevance, recency, importance

```
sims = [1.0 - d for d in distances]   # cosine -> similarity
rels = normalise(sims)               # spread across [0, 1]

for relevance, meta in zip(rels, metadatas):
    age_days = (now - meta["written_at"]) / 86_400
    recency = 0.5 ** (age_days / HALF_LIFE_DAYS)

    score = (0.6 * relevance
             + 0.25 * recency
             + 0.15 * meta["importance"])
```

- Similarity alone cannot tell a stale fact from a fresh one
- **Normalise first**: raw cosine spans 0.03-0.38 here, recency spans 0-1 — blend them raw and recency wins every time
- Inject the top five, not everything you hold



Demo — Semantic recall and the writer

Switch to terminal: python demo/demo.py — Parts 2-3



Compress

Compaction, not truncation

✂ Truncation loses the decisions

- History is the one component with no ceiling — turn 50 resends turns 1-49.
- Dropping the oldest messages is the obvious fix and the wrong one.
- The oldest turns are **where the decisions were made**.
- *"Charge it to the ship's account"* was agreed on turn 3.
- Truncate turn 3 and the agent starts asking for payment again.

Name what must survive

```
class Anchor(BaseModel):
    kind: Literal["decision", "open_task", "constraint"]
    text: str


class Compaction(BaseModel):
    summary: str = Field(
        description="What was discussed, under 120 words"
    )
    anchors: list[Anchor] = Field(
        description="Decisions, open tasks, and constraints"
    )
```

- → Do not hope the summary keeps the important parts — extract them by name
- → Anchors are structured, so they can be re-emitted verbatim next time

The compaction pass

```
def compact(messages, *, budget, keep_recent, client):
    if count_tokens(messages) <= budget:
        return messages

    system, rest = messages[0], messages[1:]
    split = safe_split(rest, len(rest) - keep_recent)
    old, recent = rest[:split], rest[split:]

    result = summarise(old, client)        # -> Compaction
    block = render_block(result)           # summary + anchors
    return [system, {"role": "system", "content": block},
            *recent]
```

- → Budget-triggered: no model call until you are actually over
- → Recent turns stay verbatim; the old span becomes one block

Three ways compaction breaks

Orphaned tool pairs

A `function_call` without its output is an API error. `safe_split` walks the boundary back until it is not between a call and its result.

A broken cache prefix

Prompt caching keys on the longest identical prefix. Leave index 0 byte-identical and insert the block after it.

Summary-of-summary drift

Compact a compaction repeatedly and detail bleeds away. Re-emit the anchors verbatim at every pass.



Demo — Compaction

Switch to terminal: python demo/demo.py — Part 4



Memory as a service

Both halves, behind MCP

Memory and compaction as MCP tools

```
mcp = MCPServer("Cantina Memory")


@mcp.tool()
def remember(patron: str, text: str, kind: str) -> str:
    """Store a durable fact about a patron."""


@mcp.tool()
def recall(patron: str, query: str, k: int = 5) -> str:
    """Recall the memories most relevant to a query."""


@mcp.tool()
def compact_thread(budget_tokens: int = 1200) -> str:
    """Compact the thread, preserving anchors."""
```

- → Memory is state behind a small, stable interface — a service, not a library
- → Any agent connects over stdio and gets the same memory, no imports
- → The store outlives any single agent process



Demo — Memory agent

Switch to terminal: python demo/demo.py — Part 5



Putting it all together

Summary and exercises

Summary

Budget the window

Count the tokens, know which component is eating them. A full window reads worse than a selective one.

Supersede, do not append

A new fact that contradicts an old one replaces it. Soft-delete keeps the audit trail, not the conflict.

Recall semantically, then score

Embed, filter by namespace, rank on relevance, recency, and importance. Decay is a weight, not a delete.

Compact, do not truncate

Preserve decisions, open tasks, and constraints by name. Keep tool pairs intact and the cache prefix alone.

Exercises

01 — Memory store: semantic recall with scoring, namespacing, supersede and forget

02 — Compaction: budget-triggered compaction that preserves anchors and tool pairs

03 — Memory MCP server: memory and compaction as tools, agent connected via stdio



Module 7 — Complete

Next: structured workflows